

Intelligent Systems

Dr. Manas Ranjan Patra



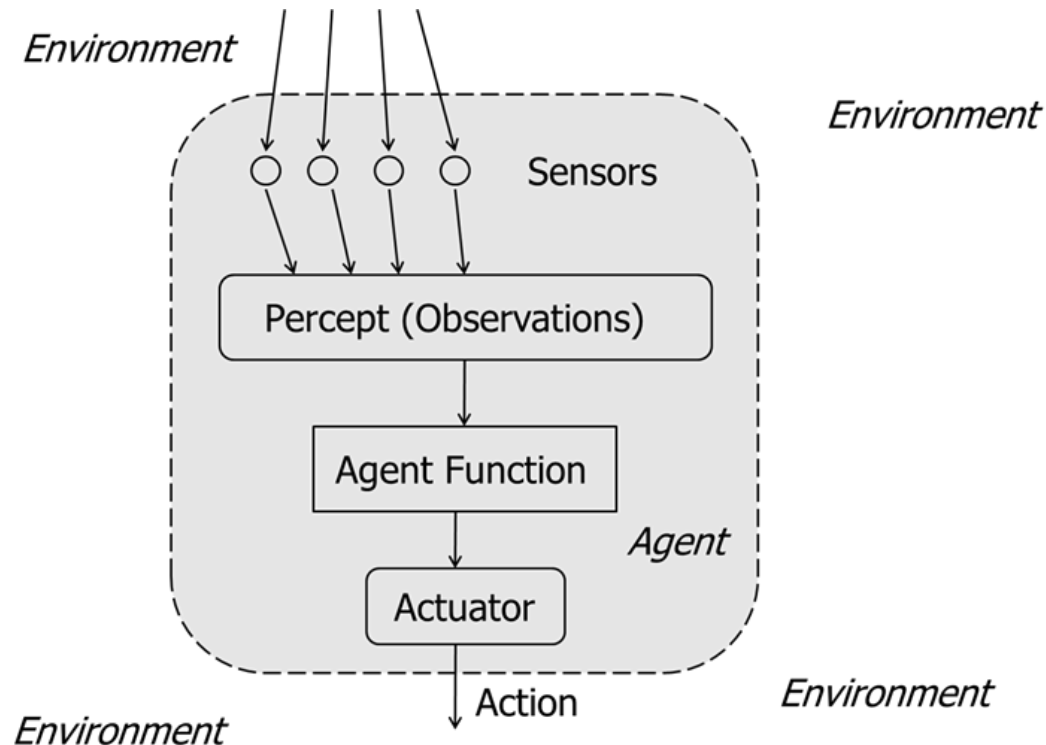
Human Cognition

Sensation
(External Stimuli)

Conception
(Internal mental events)

Association
(Rules that associate
sensations with conception)

An Intelligent agent (**Robot/Softbot**) is one which can **perceive** its environment through sensors and **act** upon the environment through **effectors/actuators**.



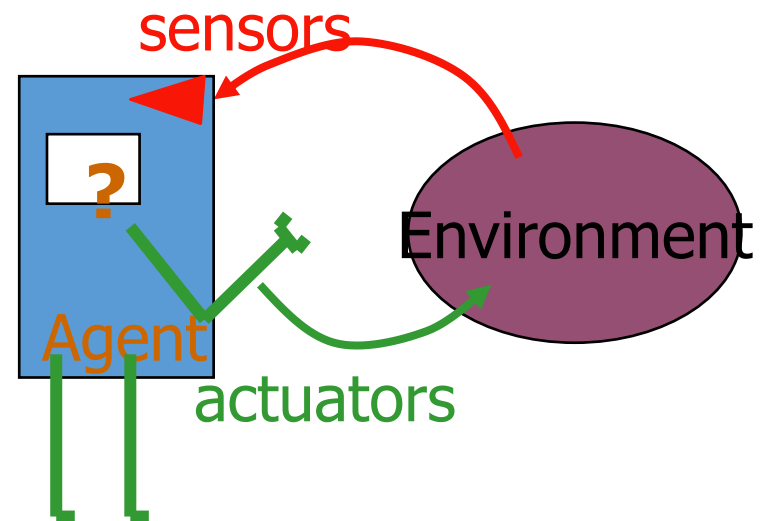
Characteristics of Intelligent agents

- **Autonomy** (Exhibits autonomous behavior)
- **Interaction** (Interacts with other agents and its environment)
- **Adaptability** (Adapts to changes in its environment)
- **Pro-activity/Goal-directedness** (Exhibit Pro-active (goal-directed) behavior)
- **Rationality** (Exhibits Rational behavior - doing the right thing)

Intelligent Agents

Definition: An agent is an encapsulated computer system that is situated in some environment and is capable of flexible, autonomous action in that environment in order to meet its design objectives.

An **Intelligent Agent** perceives its environment via **sensors** and acts rationally upon that environment with its **actuators**.



Agent attributes

- Autonomous control
- Ability to perceive its environment
- Ability to adapt to changes
- Pursue its goals
- Persists over a prolonged time

An agent should try to achieve the best outcome (even in the presence of uncertainty) with what it knows about the problem situation and its own capabilities.

Agent categories

- A **Software Agent (SA)** (**softbot**) is a software object that acts on behalf of another software object (its client) and behaves to some extent **autonomously** (without continuous direction), **socially** (interacts with other agents), **proactively** (influences its environment), and **reactively** (is influenced by its environment).
- An **intelligent software agent** is a software agent with a knowledge base with capabilities like cognition, evaluation, goals, exhibits rational behavior while making decisions, acts to attain its goals, and also **learns**.
- **Robot is a hardware agent** which is a mechanical device controlled by software (Robot control program)

Contd..

- A **Static Agent** is one that does not move from the platform that creates it.
- A **Mobile Agent** can move across a network from one physical computer to another. It can do this autonomously, as it perceives the need for such movement. It takes its run-time environment with it wherever it goes. It can stop program execution on one computer, move to another computer and begin to execute again, interacting with that computer to communicate and/or gather data.

Agent Perception & Action

- **Percept**

- Refers to what an agent has perceived from the input it received at any given instant.

- **Percept sequence**

- Refers to the complete history of what an agent has perceived so far.

- **Action**

- An agent's choice of action at any given instant which depends on the entire percept sequence observed till that time.

Agent and its Environment

Mathematically,

- The behaviour of an agent is described by an **agent function** that maps any given percept sequence to an action

$$[f: \mathcal{P}^* \rightarrow \mathcal{A}]$$

- An **agent function** is implemented by an **agent program**
- The **agent program** runs on a physical machine **architecture**

Agent = Architecture + Program

Ex: Vacuum-Cleaner world with just two locations (squares A & B)

- **Percepts:**

location and contents

e.g., [A,Dirty]

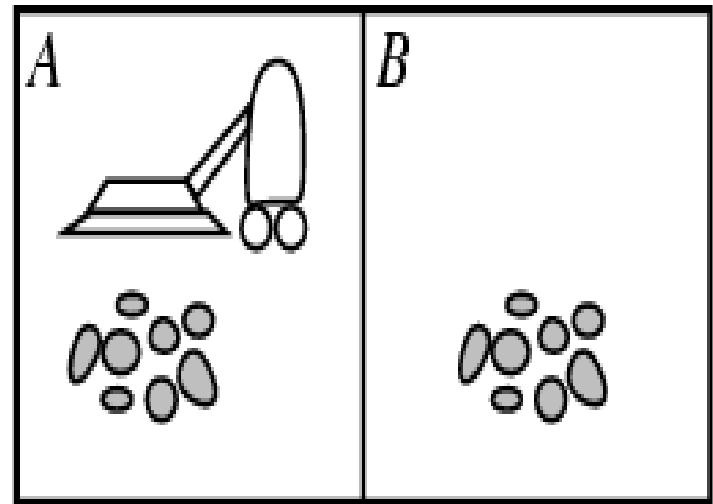
- **Actions:**

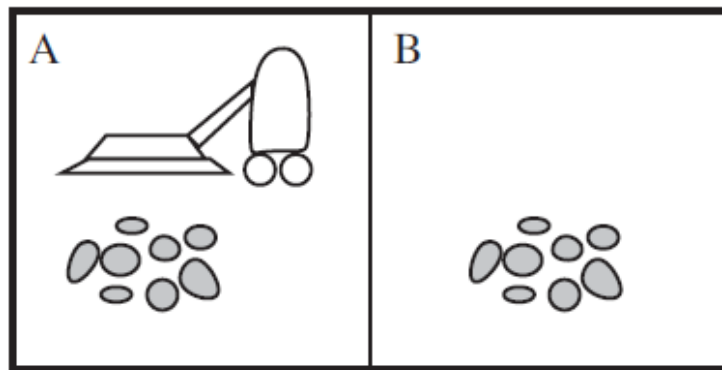
move_Left

move_Right

Suck_up_dust

no_operation





Percept sequence	Action
$[A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Dirty}]$	<i>Suck</i>
$[B, \textit{Clean}]$	<i>Left</i>
$[B, \textit{Dirty}]$	<i>Suck</i>
$[A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$

Rational Agents

- An agent should strive to "do the right thing", based on what it can perceive and the actions it can perform.
- **Performance measure:** An objective criterion measures the success of an agent's behavior

E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.

Rationality

Rationality at any given time depends on:

- agent's prior knowledge of the environment
 - actions that the agent can perform
 - agent's percept sequence till that moment
 - the performance measure that defines the criterion of success
-
- **Definition of a Rational Agent:** For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Requirements for a Rational agent

- Ability to learn
- Ability to adapt
- Autonomy in action

An agent is **autonomous** if its behavior is determined by its own experience (with ability to learn and adapt)

Building of an agent

PEAS:

- Performance measure
- Environment
- Actuators
- Sensors

PEAS

- Example: **Designing an automated driver**
 - **Performance measure:** Safe, fast, comfortable trip, maximize profits
 - **Environment:** Roads, other traffic, pedestrians, customers
 - **Actuators:** Steering wheel, accelerator, brake, signal, horn
 - **Sensors:** Cameras, sonar, speedometer, GPS, engine sensors etc.

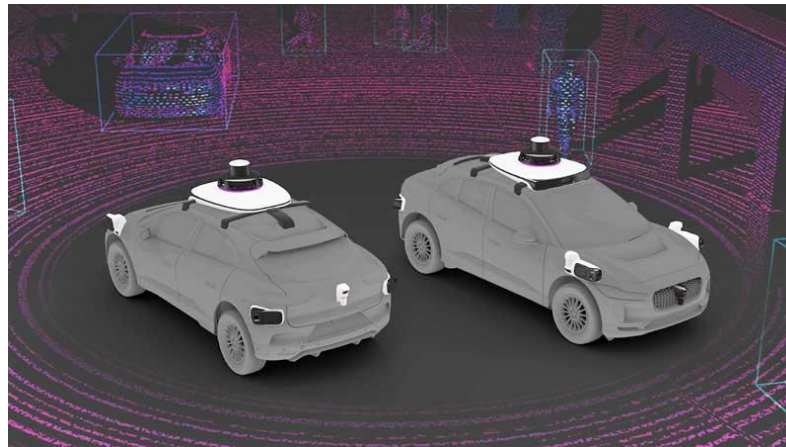
Agent Environment



Robot Planning



Waymo (Google's self-driving car)



Gaming

Environment & its characteristics

- **Fully observable** (vs. **partially observable**): An agent's sensors give it access to the complete state of the environment at each point in time.
- **Deterministic** (vs. **stochastic** - that follows a random probability distribution): The next state of the environment is completely determined by the current state and the action executed by the agent.
- **Episodic** (vs. **sequential**): The agent's experience is divided into "episodes" (each episode consists of the agent perceiving once and then performing a single action), and the choice of action in each episode depends only on the episode itself.

Environment types

- **Static** (vs. dynamic): The environment is unchanged while an agent is deliberating or it is changing.
- **Discrete** (vs. continuous): A limited number of distinct, clearly defined percepts and actions or continuous percepts and actions.
- **Single agent** (vs. multiagent): An agent operating by itself in an environment or operating in the presence of other agents.

The real world is (of course) partially observable, stochastic, dynamic, continuous, and multi-agent.

Building Intelligent Agent Systems

- An agent has some **internal data structures** that is updated as and when new **percepts** arrive.
- These data structures are operated on by the agent's **decision-making procedures** to generate an action choice, which is then passed to the **architecture for execution**.

Skeleton agent program

```
function SKELETON-AGENT(percept) returns action  
  static: memory, the agent's memory of the world  
  
  memory  $\leftarrow$  UPDATE-MEMORY(memory, percept)  
  action  $\leftarrow$  CHOOSE-BEST-ACTION(memory)  
  memory  $\leftarrow$  UPDATE-MEMORY(memory, action)  
  return action
```

A Simple Table-lookup agent

```
function TABLE-DRIVEN-AGENT(percept) returns action  
  static: percepts, a sequence, initially empty  
           table, a table, indexed by percept sequences, initially fully specified  
  
  append percept to the end of percepts  
  action  $\leftarrow$  LOOKUP(percepts, table)  
  return action
```

Drawbacks:

- constructing an explicit lookup table is difficult
- Takes a long time to build the table

The more restricted the environment, the easier the design problem. But in reality the environment is not a restricted one.

Model of different types of agents

In order of increasing generality:

- Reactive/ Reflex agents
- Deliberative/Model-based Reflex agents
- Goal-driven agents
- Utility-driven agents
- Learning agents

Reactive/Reflex agents

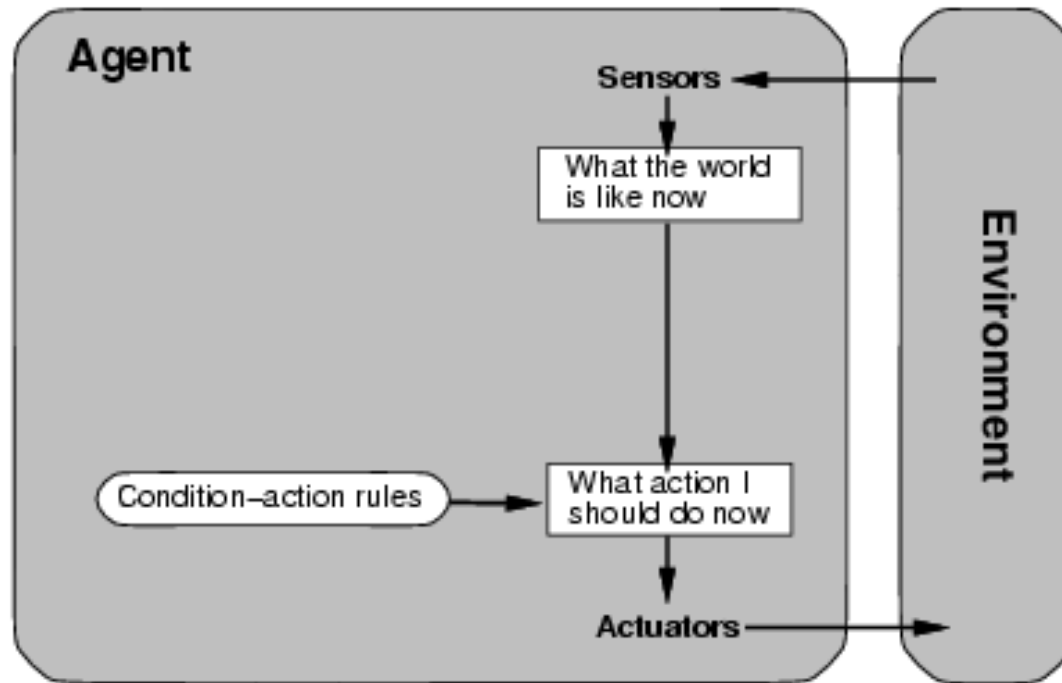
- Reactive agents follow **stimulus-response** paradigm.
- They respond to the present circumstance by following a predetermined set of condition-action rules or a mapping function
- They are devoid of any kind of memory or thinking ability.

condition–action rule/situation–action rules/if–then rules.

e.g., **if** *car-in-front-is-braking* **then** *initiate-braking*

- It finds a rule whose condition (LHS) matches the current situation (as defined by the percept) and does the action associated with that rule.
- Works on the current state only and not on the previous states
- Suitable for fully observable environment

Reactive/Reflex Agent structure



function SIMPLE-REFLEX-AGENT(*percept*) **returns** *action*

static: *rules*, a set of condition-action rules

state $\leftarrow$ INTERPRET-INPUT(*percept*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ RULE-ACTION[*rule*]

return *action*

- Ex:**
1. if it is dark then switch on the light
 2. interactive voice response (IVR)

function REFLEX-VACUUM-AGENT([location, status]) **returns** an action

if status = Dirty **then return** Suck
else if location = A **then return** Right
else if location = B **then return** Left

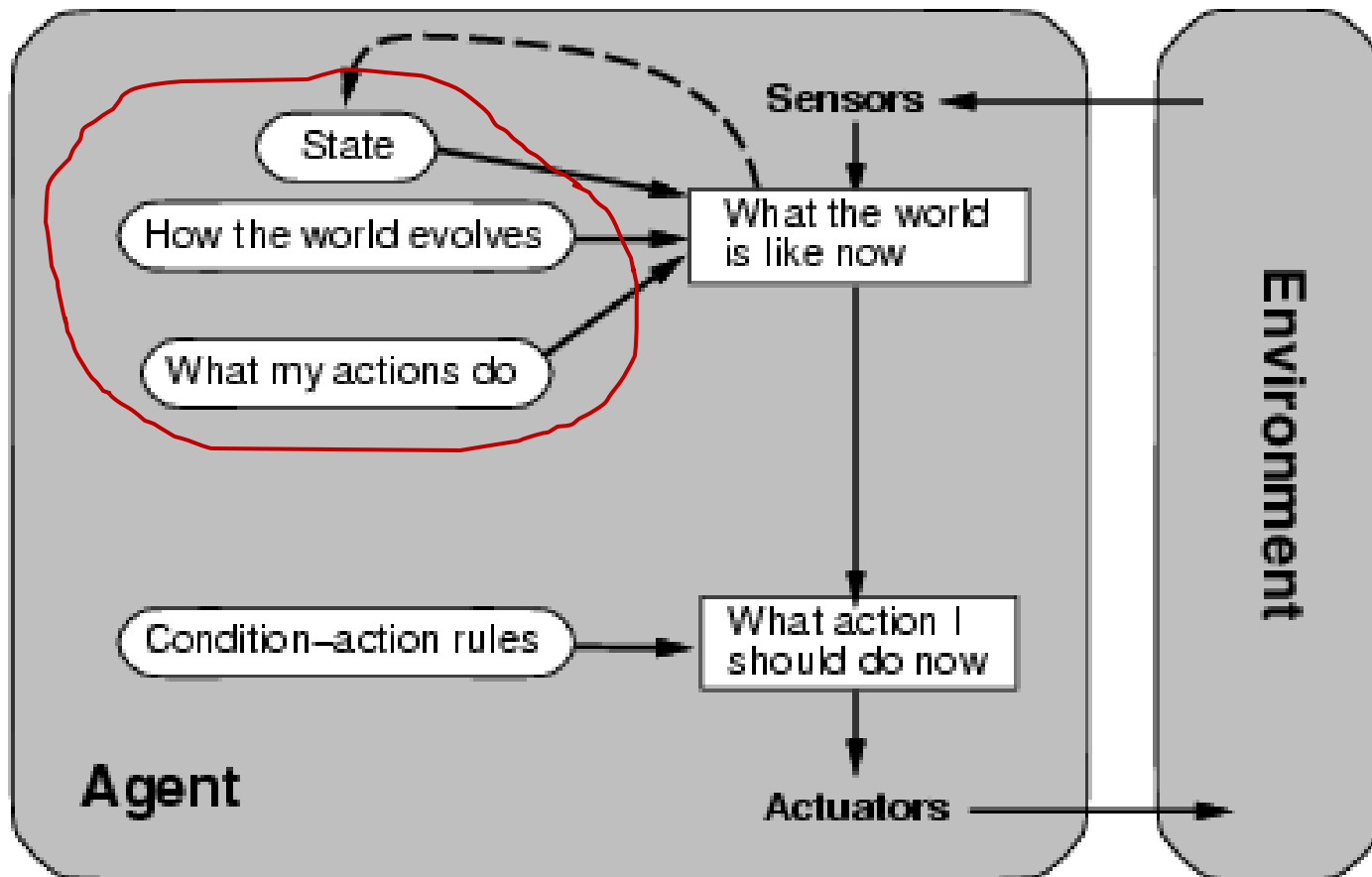
Limitation

- Difficult to list out all possible rules.
- If the environment changes, rules may have to be changed frequently

Deliberative/Model-based reflex agents

- These agents maintain an internal state which is updated whenever there is change in the environment.
- They have the ability to **reason, evaluate options, strategize, and make choices** depending on their understanding and the expected outcomes of their choices.
- They use **search algorithms, heuristics, and knowledge-based systems** to choose the best course of action depending on their objectives and the prevailing condition of the environment.
- **Suitable for partially observable environment**

Deliberative/Model-based reflex agent structure



```
function REFLEX-AGENT-WITH-STATE(percept) returns action  
  static: state, a description of the current world state  
          rules, a set of condition-action rules  
  
  state  $\leftarrow$  UPDATE-STATE(state, percept)  
  rule  $\leftarrow$  RULE-MATCH(state, rules)  
  action  $\leftarrow$  RULE-ACTION[rule]  
  state  $\leftarrow$  UPDATE-STATE(state, action)  
  return action
```

Examples:

- intelligent personal assistants
- self-driving cars
- decision-support systems
- industrial control system
- computer games like Chess

Reactive vs Deliberative agents

Feature	Reactive Agent	Deliberative Agent
World model	No internal model	Maintains an internal model
Decision making	Based on current state & simple rules	Based on planning & reasoning
Memory	Stateless	Stateful
Response time	Fast	Slow
Adaptability	Limited	Can learn & adapt
Complexity	Simple	Complex

Limitations of Reactive & Deliberative agents:

- ✓ Actions are limited to responding to changes in the environment.
- ✓ Does not exhibit proactive/goal-driven behavior

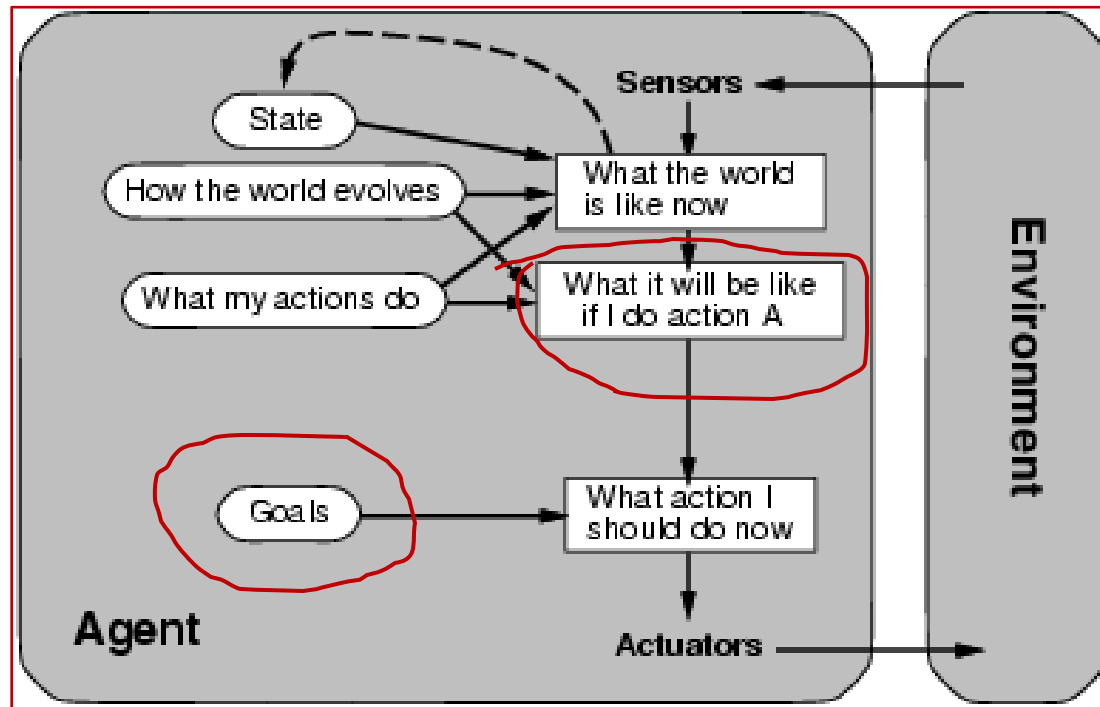
Goal-driven agents

- Knowing the current state of the environment is not always enough to decide what to do.
- A goal-driven agent is programmed to achieve a specific goal.
- An agent should be able to **plan, execute, and adjust** its actions dynamically to achieve predefined goals.
- Suitable for complex environments where flexibility and adaptability are required.

e.g., At a junction, the taxi can turn left, right, or go straight. The right decision depends on **where the taxi is trying to go** (the goal).

Goal-driven agent structure

It keeps **track of the world state** and **a set of goals** it is trying to achieve, and picks an action that will eventually lead to the achievement of its goal.



Ex: Playing chess (goal is to win)

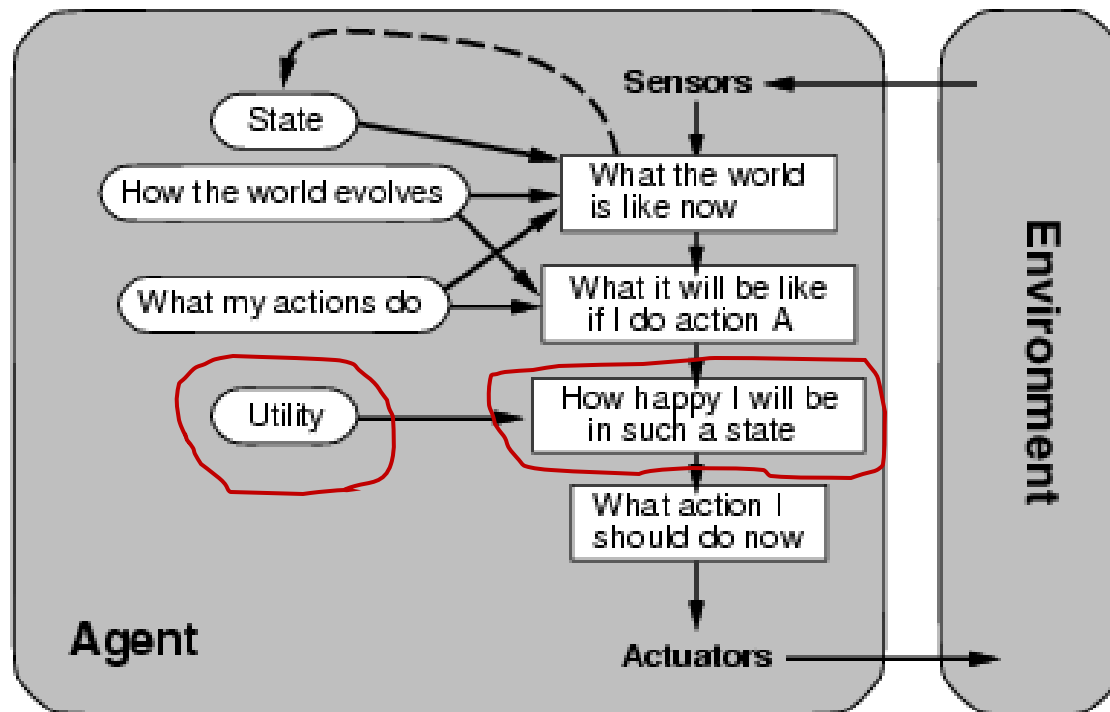
Utility-driven agents

- Goals alone are not enough to generate high-quality behavior.
- A utility function can be used to map a state onto a value which describes the effectiveness of an agent's action.

Ex: There can be many action sequences for the taxi to reach its destination (goal). But some are quicker, safer, more reliable, or cheaper than others which can be evaluated using a utility function.

Utility-driven agent structure

- It uses a model of the world, along with a **utility function that measures its preferences** among states of the world.
- Then it chooses an action that leads to the best expected utility.



Learning agents

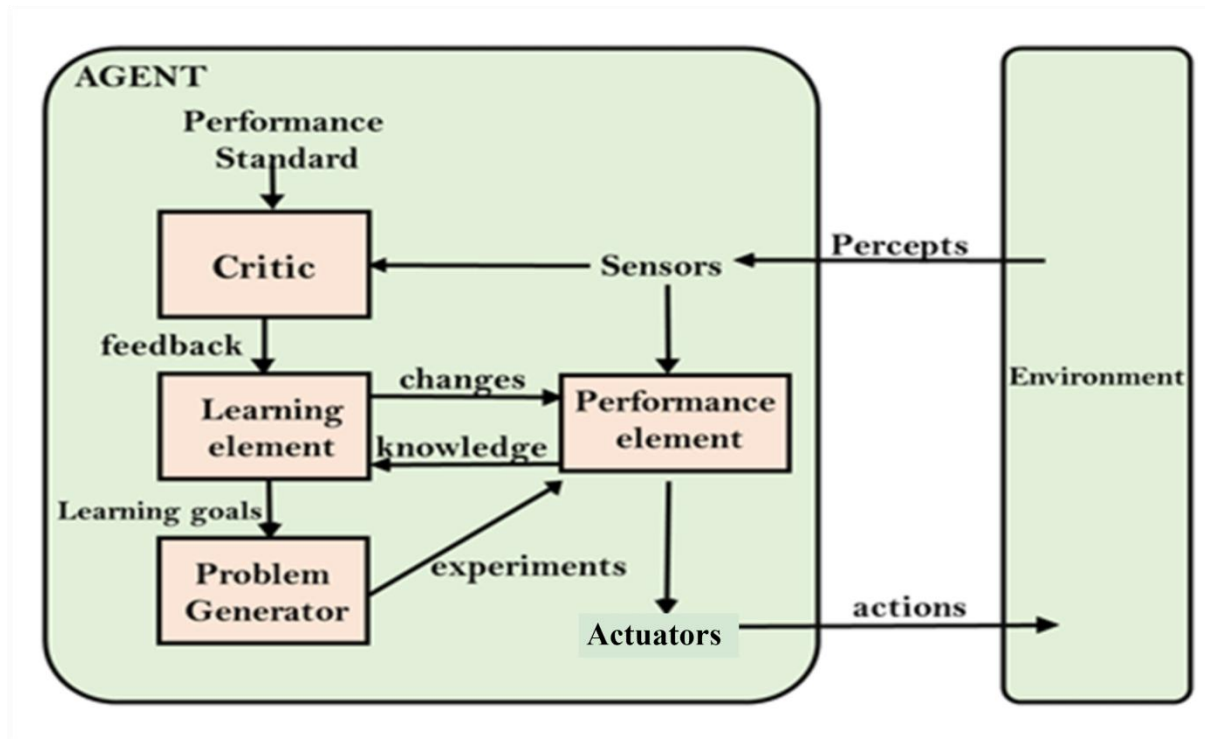
- An agent learns over a period of time from experiences & interactions with its environment.
- Machine learning: It is the ability to automatically learn from data & build models for problem-solving.
 - **Supervised** (uses labeled data)
 - **Unsupervised** (uses unlabeled data)
 - **Semi-supervised** (hybrid approach that uses a combination of both labeled & unlabeled data)
 - **Reinforcement learning** (learns using feedbacks with a reward mechanism)

A learning component consistently updates the agent's internal model/ knowledge base, and thereby improves its problem-solving ability.

Learning agent structure

Components:

- **learning element**: responsible for making improvements
- **performance element**: responsible for selecting actions
- **critic** : provides feedback
- **problem generator**: responsible for suggesting actions that will lead to new experiences



Agent Behavior: Summing -up

- It is a problem solving entity with **well-defined boundaries** and **interfaces**;
- situated in a particular environment and **acts in response to environmental inputs**;
- designed to fulfill its **design objectives** (has specific goals);
- **Autonomous** (has control both over its internal state and over its own behavior);
- capable of exhibiting **flexible problem solving behavior** while pursuing its design objectives;
 - ✓ **Reactive** behavior (able to respond in a timely fashion to changes that occur in their environment)
 - ✓ **Proactive** behavior (able to act in anticipation of future goals)